

EXPENSE TRACKER

Submitted to the

Department of Master of Computer Applications in partial fulfilment of the
requirements for the Mini Project in

Web Programming – MCA14

by

NITHIN

1MS24MC067

Under the Guidance of

Miss. Komal S. K.

RAMAIAH INSTITUTE OF TECHNOLOGY

(Autonomous Institute, Affiliated to VTU)

Accredited by National Board of Accreditation & NAAC with 'A+' Grade

MSR Nagar, MSRIT Post, Bangalore-560054

www.msrit.edu

2025



RAMAIAH
Institute of Technology

**DEPARTMENT OF MASTER OF COMPUTER
APPLICATIONS**

CERTIFICATE

This is to certify that the project entitled “**Expense Tracker**” is carried out by **Nithin – 1MS24MC067** student of 1st semester, in partial fulfillment for the Mini Project in Web Programming - MCA14, during the academic year 2024-2025.

Miss. Komal S. K.
Guide

Dr. S Ajitha
Head of the Department

Name of Examiners

Signature with Date

- 1.
- 2.

ACKNOWLEDGEMENT

With deep sense of gratitude, I am thankful to our Respected HoD, **Dr. S Ajitha** for her pillared support and encouragement. I would like to convey my heartfelt thanks to my Project Guide **Miss. Komal S. K**, Department of Master of Computer Applications, for giving me an opportunity to embark upon this topic and for her/his constant encouragement.

It gives me great pleasure to acknowledge the contribution of all people who helped me directly or indirectly realize it. First I would like to acknowledge the love and tremendous sacrifices that my Parents made to ensure that I always get an excellent education. For this and much more, I am forever in their debt.

I am thankful to all my friends for their moral and material support throughout the course of my project. Finally, I would like to thank all the Teaching and Non-Teaching Staff of Department of Master of Computer Applications for their assistance during various stages of my project work.

Nithin

ABSTRACT

This HTML-based Expense Tracker application provides a user-friendly interface for managing personal expenses. It allows users to input detailed information about their expenses, such as name, amount, category, and date. The form includes various categories like Food, Travel, Shopping, Bills, and Others, giving users flexibility in classifying their expenses. Once the data is entered, the expenses are displayed in a dynamic table format, allowing users to view, edit, and delete entries. The application also supports filtering options for weekly, monthly, and category-specific views, providing users with insights into their spending habits over time. Additionally, the total expenditure is calculated and displayed, giving users an overview of their financial situation.

For visual analysis, the app integrates a bar chart powered by Chart.js, which dynamically updates to show expense trends based on the user's selected filters. The chart provides a clear, visual representation of how expenses change over time. Furthermore, users can export their expense data to a CSV file, making it easy to analyze the data externally or keep records for future reference. This application enhances financial tracking by combining interactive forms, filters, and data visualization, all wrapped in a clean and responsive Bootstrap-based layout.

Table of Contents

1.	Introduction.....	1
1.1.	Problem Definition.....	1
2.	Implementation.....	2
2.1.	Technologies Used.....	2
2.2.	Features.....	3
2.3.	Code Snippets:.....	4
3.	User Interface Screenshots.....	12
3.1.	index.html.....	15
4.	Conclusion and Future Enhancement.....	15
4.1.	Conclusion.....	15
4.2.	Future Enhancement.....	16

1. Introduction

1.1.Problem Definition

The problem addressed by the Expense Tracker application is the challenge of managing and tracking personal finances effectively. Many individuals struggle with keeping an accurate record of their daily expenses, making it difficult to monitor spending patterns, stay within budget, and make informed financial decisions. Without proper tracking, it becomes challenging to assess which categories consume the most resources, leading to unnecessary financial strain. Additionally, manually managing and categorizing expenses can be time-consuming and prone to errors..

The application seeks to solve this problem by providing a simple, intuitive tool for users to input, view, and manage their expenses in real time. With features such as category-based classification, date filters (weekly and monthly), and a dynamic table for tracking expenses, the app makes it easier for users to keep an organized record of their spending. The inclusion of visual data representation through charts and the ability to export data to CSV further enhances the user experience, providing both clarity and ease of use for managing personal finances.

Moreover, the Expense Tracker addresses the need for better financial visibility and control. Many individuals often struggle to understand where their money goes, which can result in overspending or missing opportunities to save. By allowing users to filter expenses based on specific time periods (weekly, monthly) and categories (such as Food, Travel, and Bills), the application provides a clear breakdown of spending habits. This empowers users to make more informed decisions about their finances, such as adjusting their budget or cutting down on unnecessary expenditures. The ability to visualize expenses through graphs further enhances financial awareness, giving users a tangible representation of their financial situation and helping them identify trends or patterns that might otherwise go unnoticed.

2. Implementation

The implementation of the Expense Tracker application involves creating an intuitive and interactive platform for users to manage their personal finances. It allows users to input, view, and categorize their expenses, while providing features for filtering data by time periods and categories. The application also includes dynamic data visualization using charts, making it easier for users to understand their spending patterns. Additionally, the app provides options for editing, deleting, and exporting expenses to CSV. By combining various features, the tracker offers a comprehensive solution for financial tracking and analysis.

2.1 Technologies Used

HTML5: The structure and content of the application are built using HTML5. It defines the structure of the expense tracker page, including the form inputs, table, and layout components.

CSS3: CSS3 is used to style the application, including custom styles for layout, fonts, buttons, and tables. The application uses a responsive design to ensure it is user-friendly across devices.

JavaScript: JavaScript handles the logic and functionality of the application. It is used for managing user inputs, dynamically updating the display, handling data filtering, and interacting with the chart for data visualization.

jQuery: Although not strictly necessary for this implementation, jQuery is included to simplify DOM manipulation and event handling (e.g., form submissions and dynamic table updates).

Chart.js: This JavaScript library is used to create the bar chart that visually represents the spending trends over time. Chart.js provides easy-to-use functionality to create interactive and customizable charts.

Bootstrap 4: Bootstrap is used for responsive design, allowing the application to look good on different screen sizes. It also provides pre-built components like forms, buttons, and tables, speeding up the development process.

2.2 Features

Expense Input Form: Users can input details about their expenses, including name, amount, category, and date. The form ensures proper validation and submission of the data.

Dynamic Expense Table: The application displays a table listing all entered expenses, showing details such as the expense name, amount, category, and date. Users can view, edit, or delete any entry.

Filtering by Time Period: Users can filter expenses based on time periods like "Weekly" or "Monthly." This feature helps track expenses over different time frames to see spending patterns.

Category-Based Filtering: Users can filter expenses by categories such as Food, Travel, Shopping, Bills, and Other. This allows them to track and analyze expenses in specific areas of their life.

Total Expense Calculation: The application automatically calculates and displays the total expenses based on the current filters, giving users an immediate overview of their spending.

2.3 Code Snippets:

Index.html

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Expense Tracker</title>

  <link
href="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/css/bootstrap.min.css"
rel="stylesheet" />

  <link rel="stylesheet" href="styles.css" />

  <style>

</style>

</head>

<body>

<div class="container mt-5">

  <h2 class="text-center mb-4">💰 Expense Tracker</h2>

  <form id="expense-form" class="mb-4">

    <div class="form-row">

      <div class="col-md-3">

        <input type="text" id="expense-name" class="form-control"
placeholder="Expense Name" required />

      </div>

      <div class="col-md-2">

        <input type="number" id="expense-amount" class="form-control"
placeholder="Amount" required />

      </div>

    </div>

  </form>

</div>

</body>

</html>
```

```
</div>

<div class="col-md-2">
  <select id="expense-category" class="form-control" required>
    <option value="" disabled selected>Select Category</option>
    <option value="Food">Food</option>
    <option value="Travel">Travel</option>
    <option value="Shopping">Shopping</option>
    <option value="Bills">Bills</option>
    <option value="Other">Other</option>
  </select>
</div>

<div class="col-md-2">
  <input type="date" id="expense-date" class="form-control" required />
</div>

<div class="col-md-2">
  <button type="submit" class="btn btn-dark btn-block">Add Expense</button>
</div>
</div>
</form>

<div class="text-right mb-3" style="display: flex; justify-content: center; text-align: center;">
  <button class="btn btn-outline-dark mr-2"
onclick="filterExpenses('weekly')">Weekly</button>
  <button class="btn btn-outline-dark"
onclick="filterExpenses('monthly')">Monthly</button>
  <select id="category-filter" class="form-control ml-2"
onchange="filterByCategory()">
    <option value="">All Categories</option>
    <option value="Food">Food</option>
```

```
<option value="Travel">Travel</option>
<option value="Shopping">Shopping</option>
<option value="Bills">Bills</option>
<option value="Other">Other</option>
</select>

<button class="btn btn-outline-secondary" onclick="resetFilter()" style="margin-left: 10px;">Reset</button>

<button class="btn btn-outline-info" onclick="exportToCSV()" style="margin-left: 10px;">Export to CSV</button>
</div>

<div class="table-responsive">
  <table class="table table-bordered table-striped">
    <thead class="thead-dark">
      <tr>
        <th>Name</th>
        <th>Amount</th>
        <th>Category</th>
        <th>Date</th>
        <th>Action</th>
      </tr>
    </thead>
    <tbody id="expense-table"></tbody>
  </table>
</div>

<div class="total-expense text-right">
  <h4>Total: ₹<span id="total-amount">0</span></h4>
</div>
```

```
<div class="mt-4">
  <canvas id="expense-chart"></canvas>
</div>
</div>

<script src="https://code.jquery.com/jquery-3.5.1.min.js"></script>
<script
src="https://stackpath.bootstrapcdn.com/bootstrap/4.5.2/js/bootstrap.min.js"></
script>
<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>

</body>
</html>
```

Style.css

```
body {
  background-color: #f8f9fa;
  font-family: 'Arial', sans-serif;
}

h2 {
  font-weight: bold;
  color: #343a40;
}

.table {
  margin-top: 20px;
```

```
}
```

```
th, td {  
    text-align: center;  
}
```

```
.btn-danger {  
    background-color: #dc3545;  
    border-color: #dc3545;  
}
```

```
.btn-outline-dark, .btn-outline-secondary {  
    margin-bottom: 5px;  
}
```

```
.total-expense h4 {  
    font-weight: bold;  
    color: #28a745;  
}
```

```
canvas {  
    max-height: 300px;  
}
```

JavaScript

```
<script>  
    const expenses = [];  
    const expenseForm = document.getElementById('expense-form');  
    const expenseTable = document.getElementById('expense-table');
```

```
const totalAmount = document.getElementById('total-amount');

const expenseChartCanvas = document.getElementById('expense-
chart').getContext('2d');

const categoryFilter = document.getElementById('category-filter');

let expenseChart;
let filterType = 'all';
let selectedCategory = '';

expenseForm.addEventListener('submit', (e) => {
  e.preventDefault();

  const name = document.getElementById('expense-name').value;
  const amount = parseFloat(document.getElementById('expense-amount').value);
  const category = document.getElementById('expense-category').value;
  const date = document.getElementById('expense-date').value;

  const expense = { name, amount, category, date: new Date(date) };
  expenses.push(expense);

  updateDisplay();
  expenseForm.reset();
});

function updateDisplay() {
  let filteredExpenses = [...expenses];

  if (filterType === 'weekly') {
    filteredExpenses = filterByWeek();
```

```
    } else if (filterType === 'monthly') {
        filteredExpenses = filterByMonth();
    }

    if (selectedCategory) {
        filteredExpenses = filterByCategorySelection(filteredExpenses);
    }

    updateTable(filteredExpenses);
    updateTotal(filteredExpenses);
    updateChart(filteredExpenses);
}

function updateTable(filteredExpenses) {
    expenseTable.innerHTML = "";
    filteredExpenses.forEach((expense, index) => {
        const row = `<tr>
            <td>${expense.name}</td>
            <td>₹${expense.amount}</td>
            <td>${expense.category}</td>
            <td>${expense.date.toLocaleDateString()}</td>
            <td>
                <button class="btn btn-warning btn-sm" onclick="editExpense($
{index})">Edit</button>
                <button class="btn btn-danger btn-sm" onclick="deleteExpense($
{index})">Delete</button>
            </td>
        </tr>`;
        expenseTable.innerHTML += row;
    });
}
```

```
    });  
  }  
  
  function updateTotal(filteredExpenses) {  
    const total = filteredExpenses.reduce((sum, expense) => sum + expense.amount,  
0);  
    totalAmount.textContent = total.toFixed(2);  
  }  
  
  function deleteExpense(index) {  
    expenses.splice(index, 1);  
    updateDisplay();  
  }  
  
  function editExpense(index) {  
    const expense = expenses[index];  
    document.getElementById('expense-name').value = expense.name;  
    document.getElementById('expense-amount').value = expense.amount;  
    document.getElementById('expense-category').value = expense.category;  
    document.getElementById('expense-date').value =  
expense.date.toISOString().split('T')[0];  
  
    deleteExpense(index);  
  }  
  
  function filterByWeek() {  
    const now = new Date();  
    const oneWeekAgo = new Date(now);  
    oneWeekAgo.setDate(now.getDate() - 7);
```



```
    return expenses.filter(expense => expense.date >= oneWeekAgo);
}

function filterByMonth() {
    const now = new Date();
    const startOfMonth = new Date(now.getFullYear(), now.getMonth(), 1);

    return expenses.filter(expense => expense.date >= startOfMonth);
}

function resetFilter() {
    filterType = 'all';
    selectedCategory = '';
    categoryFilter.value = '';
    updateDisplay();
}

function filterExpenses(type) {
    filterType = type;
    updateDisplay();
}

function filterByCategory() {
    selectedCategory = categoryFilter.value;
    updateDisplay();
}

function filterByCategorySelection(expenses) {
```

```
    return expenses.filter(expense => expense.category === selectedCategory);
  }

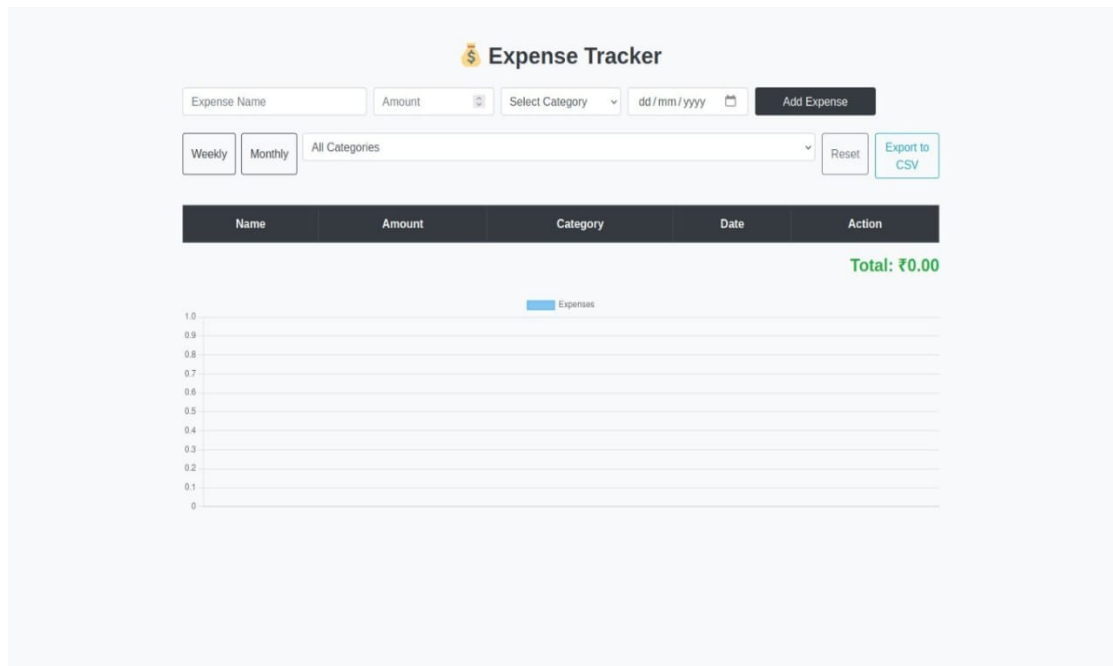
function updateChart(filteredExpenses) {
  const amounts = filteredExpenses.map(expense => expense.amount);

  if (expenseChart) {
    expenseChart.destroy();
  }

  expenseChart = new Chart(expenseChartCanvas, {
    type: 'bar',
    data: {
      labels: filteredExpenses.map(expense => expense.date.toLocaleDateString()),
      datasets: [{
        label: 'Expenses',
        data: amounts,
        backgroundColor: 'rgba(54, 162, 235, 0.6)',
        borderColor: 'rgba(54, 162, 235, 1)',
        borderWidth: 1
      }]
    },
    options: {
      scales: {
        y: {
          beginAtZero: true
        }
      }
    }
  })
}
```

```
    });  
  }  
  
  function exportToCSV() {  
    const csvRows = [];  
    const headers = ['Name', 'Amount', 'Category', 'Date'];  
    csvRows.push(headers.join(','));  
  
    expenses.forEach(exp => {  
      const row = [exp.name, exp.amount, exp.category,  
exp.date.toLocaleDateString()];  
      csvRows.push(row.join(','));  
    });  
  
    const csvString = csvRows.join('\n');  
    const blob = new Blob([csvString], { type: 'text/csv' });  
    const url = URL.createObjectURL(blob);  
    const link = document.createElement('a');  
    link.href = url;  
    link.download = 'expenses.csv';  
    link.click();  
    URL.revokeObjectURL(url);  
  }  
  
  updateDisplay();  
</script>
```

3. User Interface Screenshots



4. Conclusion and Future Enhancement

4.1 Conclusion

The **Expense Tracker** project is a comprehensive tool designed to help users manage and track their personal expenses efficiently. With features like adding, editing, and deleting expenses, users can easily monitor their spending across different categories such as Food, Travel, Shopping, Bills, and Others. The application also provides filtering options for weekly, monthly, or category-based tracking, allowing users to analyze their expenses more effectively. Additionally, the integration of Chart.js for visual representation of expenses helps users understand spending trends, while the export-to-CSV feature enables them to keep a record of their data for offline use. With a responsive design, the tracker is accessible on various devices, offering a seamless user experience. Overall, the Expense Tracker project offers a user-friendly solution for managing personal finances and making informed financial decisions.

4.2 Future Enhancement

- **Authentication and User Accounts**

Add user authentication using **JWT (JSON Web Tokens)** or **OAuth** to allow multiple users to have personalized expense data.

Each user should have their own account, and expenses should be securely stored in a database like **MongoDB**.

- **Recurring Expenses and Notifications**

Implement recurring expenses (e.g., monthly rent, utility bills) with automated reminders using **setInterval** or **Cron jobs**.

Send email or push notifications for due payments using services like **SendGrid** or **Firebase Cloud Messaging**.

- **Advanced Reporting and Insights**

Add detailed reports and charts (e.g., pie chart for category distribution) using **Chart.js**.

Include spending trends, monthly comparisons, and savings insights.

- **Multi-Currency and Budgeting**

Add multi-currency support with real-time exchange rates using APIs like **Open Exchange Rates**.

Implement a budgeting feature where users can set a monthly or category-wise spending limit.

- **Mobile App Integration**

Develop a mobile version using **React Native** or **Flutter**.

Sync data across web and mobile platforms using a **REST API** or **GraphQL**.